Gatekeeper

System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-Gatekeeper-java-spring. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-Gatekeeper-go.

Gatekeeper - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/Gatekeeper. Entry point: ./cmd/gatekeeper. Module: arrowhead/ah5-support-go/gatekeeper. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8449 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8449"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Gatekeeper	Native Go legacy-mTLS cloud discovery/negotiation engine adapted to this runtime	AH-GT-1 encryption, complete current Java edge-case parity, real peer/broker/ACL qualification

Detailed implementation evidence

1 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-Gatekeeper-rust.

Gatekeeper (Rust)

Arrowhead 5.2.2 Gatekeeper support system. Entry point src/main.rs is built as binary gatekeeper by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/gatekeeper.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/Gatekeeper, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin gatekeeper -- <config.json>

Listening port

Default 127.0.0.1:8449 (Arrowhead default, role gatekeeper). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Gatekeeper	Neighbour/relay administration, actual MQTT global discovery and negotiation with explicit cloud/organisation bindings	Exhaustive DTO/filter/failure parity and independent review

Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-Gatekeeper-java-spring/documentation/v5.2.2/Gatekeeper-v5.2.2-SysDD.tex

- Go: Eclipse-Arrowhead-go/ah5-Gatekeeper-go/documentation/Gatekeeper/Gatekeeper-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Gatekeeper/Gatekeeper-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

Gatekeeper

System Design Description

Abstract

This document describes the design and implementation of the Gatekeeper support system of Arrowhead 5.2.2 (`ah5-Gatekeeper-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 6
2.5 Local modifications 6
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the Gatekeeper system implementation, which realizes the Gatekeeper SysD [1]. The Gatekeeper connects the local cloud to its neighbour clouds through the inter-cloud MQTT relay: global service discovery of authorized foreign providers and inter-cloud negotiation of GateTunnel sessions. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation realizes the Gatekeeper SysD in `ah5-Gatekeeper-java-spring`. It follows the Arrowhead Java-Spring pattern of the other support systems. The Java sources, the private copy of the Arrowhead common library (`vendor/`), the tests, the images and the deployment definition are owned by this folder. The former native implementation (JDK `HttpsServer`, SQLite, package `eu.arrowhead.port`) has been removed. The Python predecessor is preserved in `legacy/` and is excluded from the build.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (web, security, data JPA), the vendored Arrowhead common-utils and data-transfer-objects, Bouncy Castle (PEM relay identity), and the Eclipse Paho MQTT v3 relay client. Maven produces only `arrowhead-gatekeeper-5.2.2.jar`, a Spring Boot JAR. Its entry point `GatekeeperMain` starts Gatekeeper, or with `migrate-sqlite` imports an old store.
- Storage: a private MySQL 8.4 schema `ah_gatekeeper`, through JPA (`ddl-auto=validate`) and the MariaDB driver with `sslMode=verify-full`. The application account has `SELECT`, `INSERT`, `UPDATE` and `DELETE` only.

2.2 Functional properties implementation

The following components implement the system (package `eu.arrowhead.gatekeeper`):

Component	Responsibility
`GatekeeperSystemInfo`	The registered system and services: `globalServiceDiscovery`, `interCloudNegotiation` and `gatekeeperManagement` 5.2.2, and `generalManagement` 1.0.0. The native `sysMonitor` offer is added by the common library.
`api.http.GatekeeperAPI`, `ManagementAPI`, `GeneralManagementAPI`	The HTTP operations, with the native paths, JSON shapes and status codes.
`service.FederationService`	Global service discovery and inter-cloud negotiation, on both the consumer and provider side. Provider-side `authorizationManagement.check-policies` for foreign consumers; GateTunnel sessions; the AH-GT-1 downgrade refusal.
`service.PeerNames`, `FederationSettings`	Peer cloud identities, explicit cross-organisation bindings and the federation profile (checked at start-up).
`relay.RelayClient`, `RelayTls`	The MQTT relay link with its own organisation-signed identity and CRL check. Topics `arrowhead/relay/<cloud>/gatekeeper/in/<sender CN>/<id>`, correlation and duplicate handling are byte-for-byte those of the Go and Rust implementations.
`service.ArrowheadConsumer`	ServiceRegistry lookup and orchestrated consumption through the common `ArrowheadHttpService`.
`security.*`	Token-free liveness, the caller certificate profile (`sy`, `op`), the Gatekeeper caller rules, and the native response headers and `error` field.
`jpa.*`, `migrate.*`	Entities `Cloud`, `Relay` and `Logs`, and the one-off SQLite import.

Every request passes the filter chain in this order:

1. native response headers;
2. liveness (`/health`, token-free);
3. caller certificate profile;
4. the common `OutsourcedFilter`, which verifies the identity token with Authentication and binds it to the client certificate;
5. the common management filter (whitelist policy: LocalCloudManager and the system operator);
6. the Gatekeeper caller rules (inter-cloud and echo operations: the orchestrators, LocalCloudManager, Gatekeeper and the system operator).

2.2.1 Data handled and stored

Storage: MySQL schema `ah_gatekeeper`.

Tables	Content
`cloud`	Neighbour clouds; (name, operator) is unique.
`relay`	Relays; the name is unique.
`logs`	Log entries (Log4j2 JDBC appender), served by `generalManagement.get-log`.

The relay and inter-cloud messages are JSON objects built at run time. Their structure is specified in the IDD documents.

2.3 Non functional properties implementation

2.3.1 Security

Spring Boot TLS uses a PKCS#12 system identity and truststore, with mandatory client certificates (TLS 1.3/1.2) and the cloud CRL (`arrowhead.ssl.crl-file`). Authentication is outsourced: the identity token is bound to the certificate, and only `sy` and `op` certificates are accepted. The `SecurityProfileGuard` refuses to start with TLS off, declared identities or service authorization off. `arrowhead.security.profile=development` is an explicit exception for isolated tests. The relay link uses a separate organisation-signed PEM identity and organisation trust, with CRL checking.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.3.3 Internal monitoring

Logs go to stdout, a rolling file and the `logs` table. The common `SysMonitorPublisher` publishes the `sysMonitor` observations (see the sysMonitor IDD). The echo health is degraded while the relay link is down.

2.3.4 Configuration

Configuration is `config/application.properties`. It reads the environment names of the SysD as `${ENV:default}` placeholders and the secrets through `configtree:/run/secrets/`.

2.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-gatekeeper-standalone:5.2.2` as UID 10001 with a read-only root filesystem. The local `compose.yaml` starts two services:

- Gatekeeper, on an operator-selected existing cloud network;
- its private database `gatekeeper-db` (MySQL 8.4.11, secure transport required, internal network only). The image creates the schema and restricts the application account to DML.

The HTTPS API uses port 8449. The container health check is a TCP check of that port; the native plain-HTTP health port is gone. Certificates and credentials are mounted individually. This packaging is supplied for operator deployment; existing test-cloud deployments are not changed by it.

ServiceRegistry is the bootstrap endpoint. Authentication and the orchestrator are found there, and ConsumerAuthorization and GateTunnel are consumed through orchestration on behalf of Gatekeeper. Federation additionally requires the organisation MQTT relay, configured neighbour clouds, the relay identity and organisation trust. The Compose default is AH-GT-1, with explicit legacy-mTLS compatibility; peers must agree and never silently downgrade. `migrate-sqlite <file>` imports the clouds and relays of a native SQLite store once, without starting the HTTP listener or registering.

2.4.1 Ownership and build boundary

The Arrowhead common library is a private source copy (`vendor/`, with `PROVENANCE.json`) compiled into this project. The POM inherits only the Spring Boot parent. There is no shared runtime process, data directory or Arrowhead Maven parent. Security fixes to the common library must be propagated to every vendored copy. Build with Java 21 and `mvn -B clean verify`, or use the local Docker build.

2.5 Local modifications

Notes on this implementation in the workspace:

- Inter-cloud orchestration with DynamicServiceOrchestration (local extension); provider-side `check-policies` for foreign consumers.
- Services registered with version `5.2.2`; `generalManagement` 1.0.0 with `status`, `get-log` and `get-config`.
- Deviations from the native implementation: a wrong method answers 405; trailing slashes are not matched; the plain-HTTP health port is removed. Error bodies keep the native `error` field.

2.6 Verification

The project owns these JUnit tests:

- a route contract test: every native path and method through the deployed filter chain, with the authorised, unauthenticated, wrong-certificate, refused-profile, missing-certificate and unauthorised statuses;
- the provider-side `check-policies` tests and the federation protocol steps;
- management validation and views, and peer names;
- the refusal of insecure settings.

Two integration tests run against containers: the whole application against a fresh database image (schema validation under DML-only grants, plus the SQLite import), and a relay round trip against a TLS Mosquitto broker. A live federation run against the Go and Rust Gatekeepers has not been repeated for this packaging.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`globalServiceDiscovery`	[1]	[2]	[3]
`interCloudNegotiation`	[1]	[4]	[5]
`gatekeeperManagement`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
authorizationManagement	[14]	[15]	–
gateTunnelConnection	[16]	[17]	–
inter-cloud messages	Relay (MQTT)	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “Gatekeeper – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-Gatekeeper-java-spring/documentation/v5.2.2/Gatekeeper-v5.2.2-SysD.pdf.

[2] ——, “globalServiceDiscovery – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-globalServiceDiscovery-SD.pdf.

[3] ——, “globalServiceDiscovery HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-globalServiceDiscovery-HTTP-IDD.pdf.

[4] ——, “interCloudNegotiation – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-interCloudNegotiation-SD.pdf.

[5] ——, “interCloudNegotiation HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-interCloudNegotiation-HTTP-IDD.pdf.

[6] ——, “gatekeeperManagement – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-gatekeeperManagement-SD.pdf.

[7] ——, “gatekeeperManagement HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-gatekeeperManagement-HTTP-IDD.pdf.

[8] ——, “sysMonitor – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.2.2, 2026, gatekeeper-v5.2.2-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.2.2, 2026, gatekeeper-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[11] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[14] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[15] ——, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationManagement-SD.pdf.

[16] ——, “GateTunnel – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-GateTunnel-java-spring/documentation/v5.2.2/GateTunnel-v5.2.2-SysD.pdf.

[17] ——, “gateTunnelConnection – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelConnection-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-09	5.2.2	Implementation converted to the Arrowhead Java-Spring pattern: vendored common library, private MySQL schema (JPA), common authentication, registration, orchestration and sysMonitor; relay, federation protocol and service contracts unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Gatekeeper

1.1 System overview

Cross-cloud discovery and negotiation. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

1.2 Produced and consumed services

Produced HTTPS services: `echo`, `gatekeeperManagement`, `generalManagement`, `globalServiceDiscovery`, `interCloudNegotiation`. HTTPS service version 5.2.2. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

1.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Reuses the existing native Go legacy-mTLS relay engine, adapted to pinned clients. AH-GT-1 end-to-end encryption is absent. Broker ACLs and trusted neighbour provisioning remain required; no live federation is exercised by this delivery.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

1.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

2 Implementation

Go 1.26 or newer builds the native binary. State is serialized as private JSON documents; updates clone state, validate, persist and then publish. Server timeouts and bounded message sizes constrain resource use. Persistent outboxes are at-least-once and cannot provide distributed exactly-once delivery. Detailed source, configuration and test instructions are in the repository README and verification report.

2.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete

behavioral conformance.

Method	Path	Access
GET	/gatekeeper/echo	identity
POST	/gatekeeper/init_gsd	identity
POST	/gatekeeper/init_multi_gsd	identity
POST	/gatekeeper/init_icn	identity
GET	/gatekeeper/mgmt/clouds	management
POST	/gatekeeper/mgmt/clouds	management
DELETE	/gatekeeper/mgmt/clouds/{operator}/{name}	management
GET	/gatekeeper/mgmt/relays	management
POST	/gatekeeper/mgmt/relays	management
GET	/gatekeeper/mgmt/status	management
GET	/gatekeeper/monitor/echo	certificate
GET	/gatekeeper/general/mgmt/get-config	management
POST	/gatekeeper/general/mgmt/logs	management

1 Gatekeeper

1.1 System overview

Native Rust development profile for local Java source `JavaSupportRuntime/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Neighbour and relay administration

1.5 Open release gates

AMQPS federation discovery/negotiation absent, returns501. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `echo`, `gatekeeperManagement`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/gatekeeper/general/mgmt/get-config	management
POST	/gatekeeper/general/mgmt/logs	management
GET	/gatekeeper/monitor/echo	certificate
POST	/gatekeeper/init_gsd	identity / 501
POST	/gatekeeper/init_multi_gsd	identity / 501
POST	/gatekeeper/init_icn	identity / 501
GET	/gatekeeper/mgmt/clouds	management
POST	/gatekeeper/mgmt/clouds	management
DELETE	/gatekeeper/mgmt/clouds/{operator}/{name}	management
GET	/gatekeeper/mgmt/relays	management
POST	/gatekeeper/mgmt/relays	management

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.